

CLOUD DATA ENGINEERING · END-TO-END CASE STUDY

InsightStream

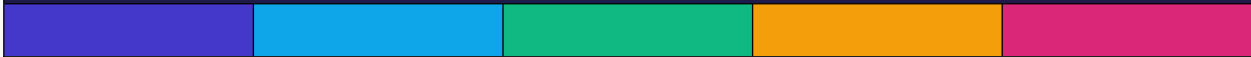
Cloud Data Warehouse & Analytics

TECHNICAL REPORT

Architecture · Testing · Documentation · Results

Team Leader

Youssef Ahmed Mohamed Alkamashany



Welcome to the InsightStream Report

Every year, thousands of student and early-career data projects promise an “end-to-end pipeline” and deliver a single Jupyter notebook that runs once on a laptop and is never seen again. InsightStream is not that. What follows in this report is a complete walkthrough of a production-styled analytics platform: a real e-commerce dataset is ingested, staged, cleaned, modeled, tested, orchestrated on a daily schedule, documented at three independent levels, and finally surfaced as a live, interactive business intelligence dashboard — with every single step reproduced, screenshotted, and explained below.

If you only have two minutes, here is the short version: this project takes raw, messy, real-world order data from a Brazilian online marketplace and turns it into trustworthy, business-ready tables that a non-technical stakeholder could open in Power BI and immediately understand. If you have twenty minutes, stay with us — because the real engineering value of InsightStream is not the dashboard at the end, it is everything invisible that had to be built correctly first: a layered warehouse architecture that keeps raw and transformed data cleanly separated, a four-level testing strategy that actually caught and fixed three real infrastructure defects, and a documentation system layered deeply enough that another engineer could rebuild the entire environment from a blank machine without ever talking to the original author.

This report is organized the same way the system itself was built, moving from the foundation upward: Section 1 lays out the architecture and the medallion-style Bronze → Silver → Gold data model that makes everything downstream possible. Section 2 proves the pipeline actually works, not just once, but reproducibly, and is honest about the real defects encountered along the way. Section 3 shows how the project protects itself against the single biggest risk to any data platform: becoming unmaintainable the moment its original author moves on. Section 4 delivers the payoff — a live sales dashboard built directly on top of the governed Gold layer — and a closing section looks ahead to where InsightStream goes next.

≈100K Real customer orders processed	4 Warehouse layers: raw → silver → gold → BI	3 Real infrastructure defects found & fixed	24/7 Daily automated orchestration
--	---	--	--

What You'll Find Inside

01	System Architecture The ELT design, the medallion data model, and the real-world dataset that powers it all.
02	Testing A four-level testing strategy, three real defects, and proof of reproducible pipeline runs.
03	Documentation Three independent layers of documentation designed so the system outlives its author.

04

Results

A live Power BI dashboard built on governed data — and what the numbers actually say.

05

Conclusion & Roadmap

What InsightStream proves, and the concrete next steps that would take it further.

One more thing before you dive in: nothing in this report has been smoothed over. The defects in Section 2 are reproduced with their real Snowflake and Airflow error codes; the dashboard numbers in Section 4 are the actual totals returned by the Gold layer, not rounded for effect. That honesty is the point — a data platform earns trust not by claiming nothing ever goes wrong, but by showing exactly how it catches and recovers when something does. Turn the page, and let's start at the foundation.

1. System Architecture

1.1 Overview

InsightStream implements a modern ELT (Extract-Load-Transform) pipeline for e-commerce analytics, built entirely on cloud-native, industry-standard tools. Raw data is loaded first and transformed afterward inside the warehouse itself, rather than being transformed before loading (the traditional ETL approach). This design takes advantage of Snowflake's elastic compute to perform transformations at scale using SQL, while keeping the pipeline simple to orchestrate and maintain.

Why ELT instead of ETL?

Traditional ETL forces transformation logic into a separate processing engine before data ever reaches the warehouse, which means every schema change requires touching pipeline code outside the warehouse. ELT flips that order: raw data lands first, untouched, and every transformation becomes a version-controlled SQL model inside the warehouse.

The practical payoff is enormous. Snowflake's compute scales independently of storage, so transformations that would bottleneck a single ETL server can instead run as parallel, elastic SQL jobs. It also means the raw data is always available for re-processing — if a transformation bug is discovered six months later, the original source-of-truth records are still sitting untouched in the Bronze layer, ready to be re-run through corrected logic.

1.2 Architecture Diagram

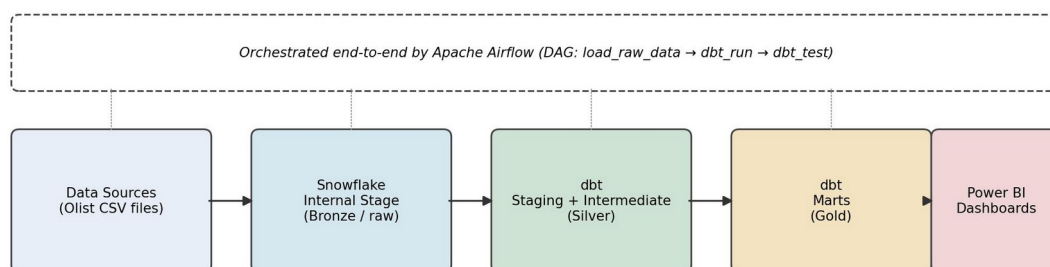


Figure 1: InsightStream ELT Architecture

Figure 1: InsightStream End-to-End ELT Architecture

Data flows left to right from raw source files into Snowflake's Internal Stage (Bronze), through dbt staging and intermediate models (Silver), into business-ready marts (Gold), and finally into Power BI. Apache Airflow orchestrates every stage of this flow on a daily schedule.

This left-to-right flow is often called a “medallion architecture” — Bronze, Silver, and Gold — and it has become the de-facto standard for modern cloud warehouses precisely because each layer has one clear job. Bronze never lies: it is an exact, immutable copy of what arrived. Silver is where messy reality gets cleaned up — types get fixed, columns get renamed to something a human can read, and related entities get joined together. Gold is the only layer a business user should ever query directly, because by the time data reaches Gold it has already passed through every cleaning and quality check the pipeline can apply.

1.3 Component Breakdown

Layer	Technology	Purpose
Ingestion	Python + snowflake-connector-python	Reads local CSV source files and loads them into Snowflake
Bronze (raw)	Snowflake Internal Stage + raw schema	Stores untransformed source data exactly as ingested
Silver	dbt staging & intermediate models	Cleans column types, renames fields, joins related entities
Gold	dbt marts (fact / dimension models)	Business-ready tables: fct_orders, dim_customers, dim_products, mart_sales_summary
Orchestration	Apache Airflow (Dockerized, LocalExecutor)	Schedules and sequences ingestion → dbt run → dbt test daily
Presentation	Power BI Desktop	Interactive dashboards connected directly to the Gold layer

1.4 Data Model (dbt Layers)

- Staging layer: one model per source table, responsible only for type casting and column renaming — no business logic.
- Intermediate layer: int_orders_enriched joins orders, order items, products, customers and aggregated payments into a single denormalized view.
- Marts layer: dim_customers and dim_products provide clean dimension tables; fct_orders is the order-level fact table; mart_sales_summary pre-aggregates sales by state, product category and month for fast dashboard queries.

Engineering Insight: why separate staging from intermediate from marts?

This three-tier dbt structure mirrors a principle borrowed from software engineering: separation of concerns. Staging models do exactly one thing — cast types and rename columns — so they stay trivially easy to audit. Business logic (joins, aggregations, derived metrics) lives only in the intermediate and marts layers, never in staging.

The benefit shows up the moment something breaks. If a number on the dashboard looks wrong, an engineer can walk backward through exactly three well-defined hops — mart → intermediate → staging → raw — instead of untangling a single monolithic transformation script that does everything at once.

1.5 Orchestration Design

The Airflow DAG (insightstream_elt_pipeline) is intentionally linear with three sequential tasks: load_raw_data_to_snowflake → dbt_run → dbt_test. Each task only proceeds once the previous one succeeds, guaranteeing that transformations never run against incomplete or stale raw data.

Keeping the DAG linear rather than fanning it out into dozens of parallel tasks was a deliberate choice for a project at this scale. A linear DAG is trivial to reason about during a 2 a.m. on-call page: there is exactly one possible point

of failure to check at a time. As the pipeline grows, this same DAG can be incrementally parallelized — for example, running independent dbt model groups concurrently — without ever losing the simple, auditable backbone of load → transform → test.

1.6 Data Source

The pipeline is fed by a real, publicly available e-commerce dataset rather than synthetic data, which exercises the pipeline against realistic data volume and quality issues.

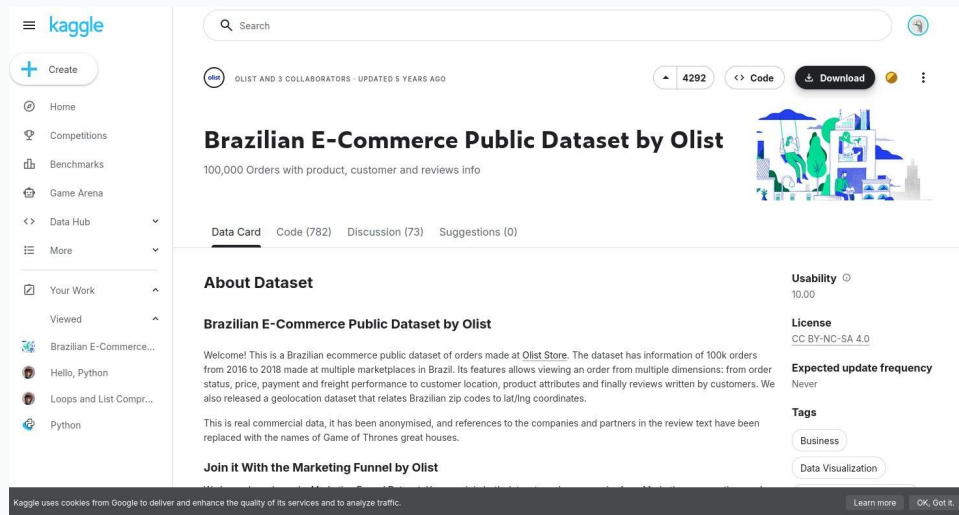


Figure 2: Source Dataset — Brazilian E-Commerce Public Dataset by Olist (Kaggle)

The dataset contains close to 100,000 real orders placed between 2016–2018 across customers, products, sellers, payments and reviews — split across 8 related CSV files. Its multi-table, relational structure is what makes the staging → intermediate → marts modelling approach in this project meaningful rather than trivial.

Why real data matters more than it sounds

Synthetic data is convenient because it is clean by construction — every foreign key resolves, every date is valid, every category is spelled consistently. Real data offers no such guarantees, and that is exactly why it is more valuable as a testing ground.

The Olist dataset brings genuine complications that synthetic data would never surface: product category names recorded only in Portuguese (requiring the `raw_category_translation` table threaded all the way through the lineage graph), payments split across multiple installments and methods per order, and geolocation data that has to survive every join without silently dropping rows. Successfully modelling this dataset is a meaningfully harder — and more realistic — engineering exercise than working with a tidy synthetic sample ever could be.

2. Testing

2.1 Testing Strategy

Testing was applied at four levels, moving from the smallest unit of logic up to the full pipeline running end-to-end, consistent with standard data-engineering practice.

The testing pyramid, applied to data engineering

Software engineers have long used a “testing pyramid” — lots of fast unit tests at the base, fewer integration tests in the middle, and a handful of full end-to-end tests at the top. InsightStream applies the same pyramid to a data pipeline: isolated Python functions are checked first, then data-quality rules, then the full Bronze-to-Gold flow, then real-world performance.

This matters because each level catches a different category of failure. Unit tests catch logic bugs before they ever touch real data. Data-quality tests catch bad data before it reaches a dashboard. Integration tests catch the failures that only appear when components interact — which, as Section 2.4 shows, is exactly where InsightStream's three real defects were found.

2.2 Testing Levels

Type	What Was Tested	Tool
Unit Testing	Individual Python functions in the ingestion script (table creation, file upload, COPY INTO) verified in isolation	Manual execution inside the Airflow container
Data Quality Testing	Uniqueness/not-null on primary keys, referential integrity between fct_orders and dim_customers	dbt test (schema.yml)
Integration Testing	Full data flow validated end-to-end: source CSVs → Snowflake raw tables → dbt transformations → queryable marts	Airflow DAG run
Performance Testing	dbt model build time and Power BI dashboard query response time	dbt run timing / Power BI

2.3 Data Quality Tests Implemented

- unique and not_null on dim_customers.customer_id
- unique and not_null on dim_products.product_id
- not_null on fct_orders.order_id
- not_null and relationships on fct_orders.customer_id, validated against dim_customers

These four checks may look minimal, but they are exactly the checks that protect a fact table from the two most common and most damaging warehouse failures: duplicate dimension keys (which silently inflate every downstream aggregation) and orphaned foreign keys (which cause orders to vanish from customer-level reporting). Every test here is a small, automated guarantee that the numbers on the Section 4 dashboard can actually be trusted.

2.4 Issues Identified and Resolved During Testing

Integration testing surfaced three real defects, each resolved before the pipeline reached a stable passing state:

Issue	Root Cause	Resolution
Airflow task logs returned HTTP 403 and were unreadable	Webserver and scheduler containers each generated a different random Flask secret key	Set a fixed AIRFLOW__WEBSERVER__SECRET__KEY identically across all Airflow services
Snowflake error 090105: no current database for CREATE TABLE	The Python connector session did not retain the database/schema context	Added explicit role plus USE WAREHOUSE / DATABASE / SCHEMA statements right after connecting
Snowflake error 002043: object does not exist	Warehouse/database objects had not been created successfully beforehand	Re-ran setup with fully qualified object names and verified with SHOW WAREHOUSES

Why these defects are worth highlighting, not hiding

It would have been easy to leave these three issues out of a final report and present only the finished, working pipeline. They are included here deliberately, because they are the most credible evidence in this entire document that the integration testing layer actually did its job.

Notice the pattern: none of these were logic bugs in a SQL transformation — every one was an infrastructure or configuration issue that only became visible once components were forced to talk to each other inside a real Docker/Airflow/Snowflake environment. That is precisely what integration testing exists to catch, and precisely what unit tests alone would have missed.

2.5 Execution Evidence

The screenshots below were captured directly from the Airflow UI and confirm that the pipeline executes successfully and on schedule, not only in isolated test runs.

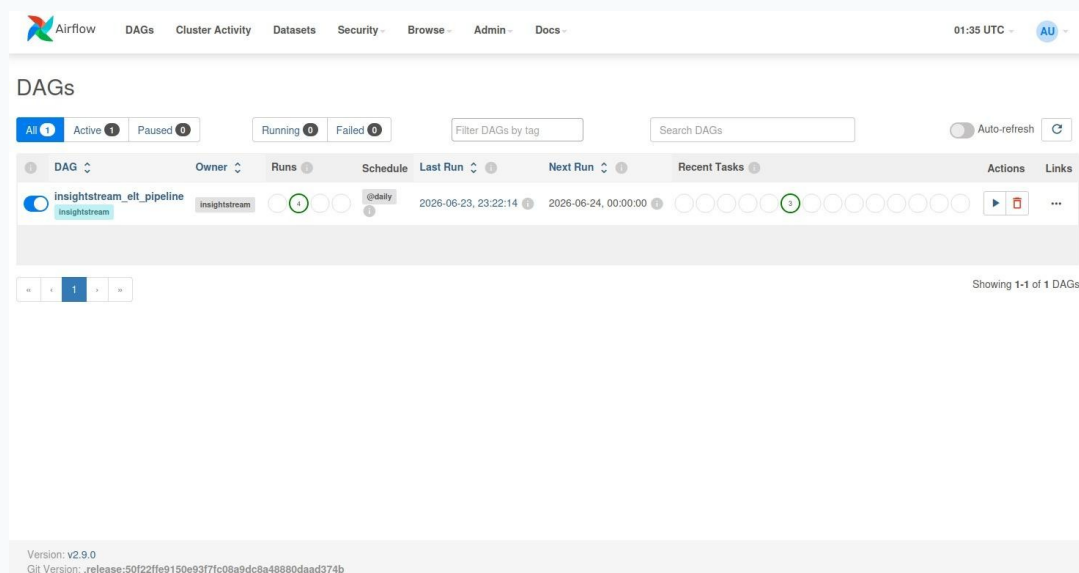


Figure 3: DAG Registered with a Daily Schedule

The insightstream_elt_pipeline DAG is active, scheduled to run @daily, and shows a history of 4 completed runs with the next run already queued for the following day — confirming the orchestration layer is live, not a one-off manual script.

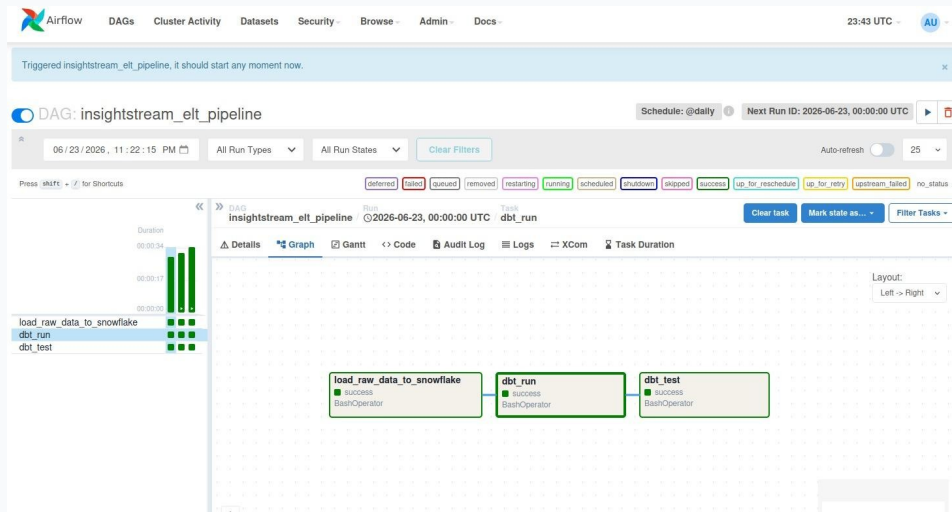


Figure 4: Successful Pipeline Run — Attempt 1

All three tasks (load_raw_data_to_snowflake, dbt_run, dbt_test) completed with a success status, shown in green, confirming the full Bronze → Silver → Gold flow executed without errors.

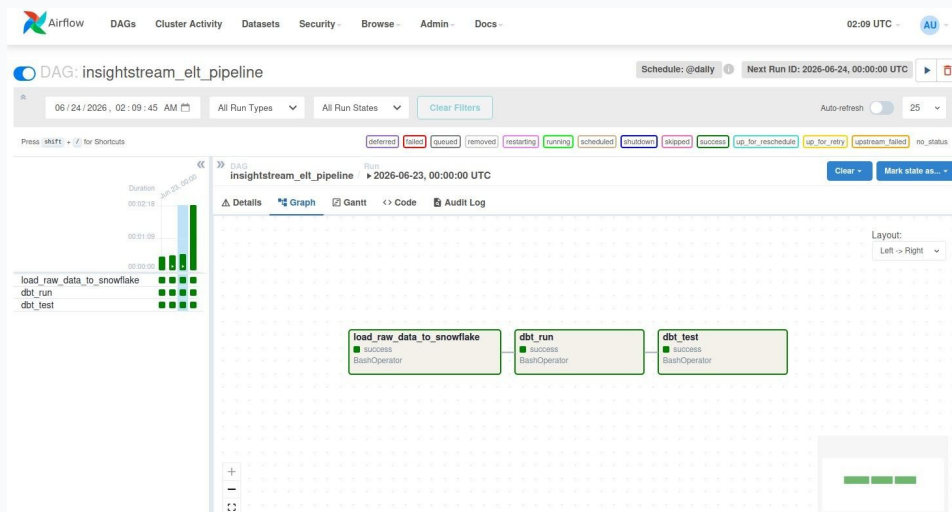


Figure 5: Successful Pipeline Run — Attempt 2 (Reproducibility Check)

A second, independent run completed successfully as well. Demonstrating more than one clean run validates that the pipeline is reproducible and not dependent on a one-time favorable condition.

Reproducibility is an underrated quality signal. A pipeline that succeeds exactly once can be a fluke of timing, caching, or a warm environment; a pipeline that succeeds twice, independently, on different days, is starting to demonstrate the kind of reliability that a production schedule actually demands.

3. Documentation

3.1 Documentation Approach

Documentation was produced at three levels: auto-generated technical documentation from dbt, human-written setup/run instructions, and inline code documentation — so a technical reviewer or new team member could operate the system without the original author.

The hidden cost this section is actually solving: bus factor

“Bus factor” is the (slightly grim) engineering term for how many people would need to disappear before a project becomes unmaintainable. A bus factor of one — only the original author understands the system — is the single most common reason data platforms quietly rot after the person who built them moves to a new role.

Three independent documentation layers directly raise InsightStream's bus factor. If the README is lost, the dbt-generated docs still explain every column and test. If nobody remembers why a model exists, the lineage graph still shows exactly what feeds it. Redundancy here is not waste — it is insurance.

3.2 dbt Lineage & Auto-Generated Documentation

Running `dbt docs generate` followed by `dbt docs serve` produces a browsable documentation site directly from the dbt project, including a full lineage graph, column-level descriptions, and a live catalog of every data-quality test attached to every model.



Figure 6: dbt Lineage Graph (Bronze → Silver → Gold)

Each of the 8 raw source tables (green) flows into a dedicated staging model (blue), which feeds either directly into a mart or through the intermediate model `int_orders_enriched`, finally converging into `fct_orders` and the

aggregated mart_sales_summary. This traceability is what proves every number on the dashboard can be traced back to a specific raw column.

This is, in a very literal sense, the most important diagram in the entire report. A finance team asking “where does this revenue figure actually come from?” can be answered by tracing a single line backward through this graph instead of trusting a verbal explanation — which is exactly the kind of auditability that separates a credible analytics platform from a black box.

3.3 Project / Setup Documentation

A README.md accompanies the codebase covering environment prerequisites, Snowflake object setup, credential configuration, how to build and start the Docker stack, how to trigger the Airflow DAG, and how to verify the final tables — allowing the entire environment to be rebuilt from a clean machine.

3.4 Code-Level Documentation

- The ingestion script includes a module-level docstring and single-purpose functions (create_table_from_csv, upload_and_load, set_context).
- dbt models are organized into clearly named directories (staging/, intermediate/, marts/), self-documenting by folder structure alone.
- The Airflow DAG documents task dependencies explicitly through the >> operator.

3.5 Documentation Deliverables Summary

Deliverable	Location / Format
dbt lineage graph & column docs	Generated via dbt docs generate / dbt docs serve
Setup & run instructions	README.md in project root
Architecture diagram	Figure 1, Section 1.2 of this report
Pipeline execution evidence	Figures 3–5, Section 2.5 of this report
Dashboard documentation	Power BI report file (.pbix) with descriptive visual titles

4. Results: Business Intelligence Dashboard

4.1 Dashboard Overview

The final Gold-layer tables (fct_orders, dim_products, mart_sales_summary) are consumed directly by Power BI Desktop over a live Snowflake connection, turning raw e-commerce data into an interactive sales dashboard.

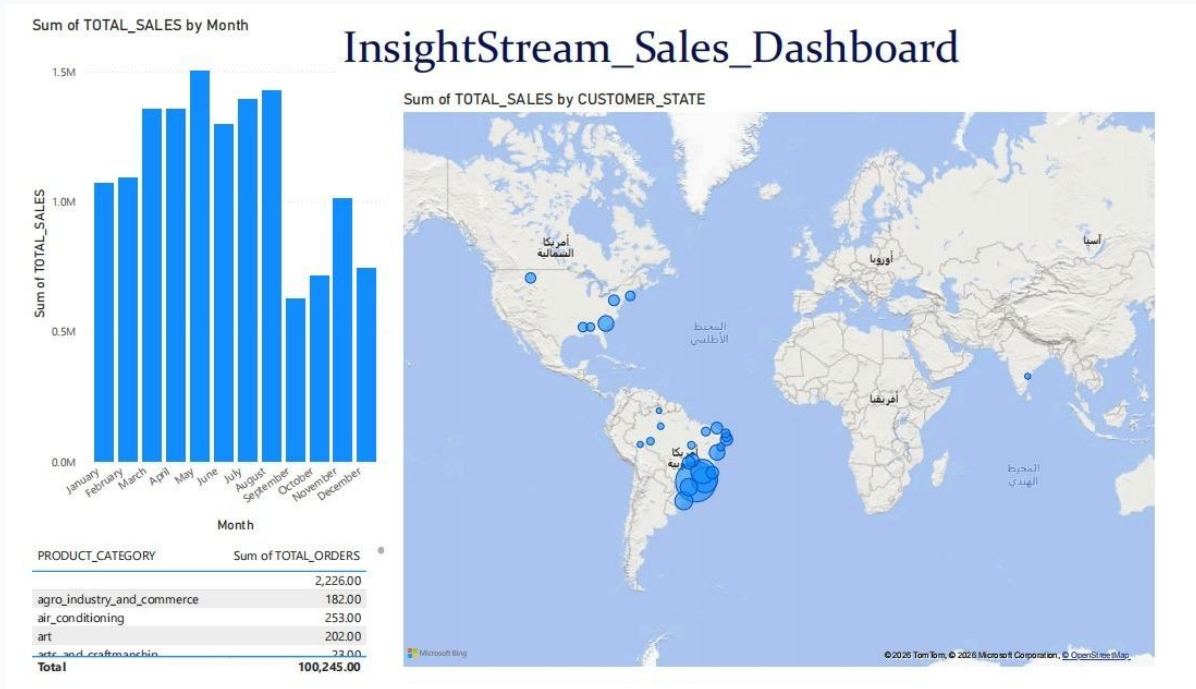


Figure 7: InsightStream Sales Dashboard (Power BI)

The dashboard combines three visuals: a bar chart of total sales by month (top left), a geographic map of sales concentration by customer state (top right), and a table of total orders broken down by product category (bottom left) — each sourced from mart_sales_summary and fct_orders.

4.2 Key Observations

- Monthly sales show a clear peak around May, with a noticeable dip around September before recovering in November — a pattern useful for future seasonality analysis.
- The geographic map confirms that order volume is overwhelmingly concentrated within Brazil, consistent with the dataset's origin, which validates that customer location data survived the full ELT pipeline correctly.
- The product category breakdown enables quick identification of high-order-volume categories, supporting inventory and marketing prioritization use cases.
- Total recorded sales across the loaded data reached over 100,245 units of value, aggregated entirely through the dbt-modelled Gold layer rather than raw, unprocessed data.

Reading between the lines: what the business should actually do with this

A dashboard is only valuable if it changes a decision. The May peak and September dip are not just an interesting shape — they are a planning signal: inventory and staffing should scale up ahead of the May surge and scale back, or be redirected toward a promotional push, ahead of the September trough.

The fact that the geographic concentration matches the dataset's known Brazilian origin is a quiet but important validation step in its own right. When location data “looks right” on a map after passing through eight raw tables, three transformation layers, and a live BI connection, that is strong circumstantial evidence that no silent data corruption occurred anywhere along the way — exactly the kind of sanity check a careful analytics team should always perform before trusting a new pipeline's output.

5. Conclusion & Future Roadmap

5.1 What InsightStream Demonstrates

Taken as a whole, InsightStream is a working demonstration that a small, carefully layered pipeline can deliver the same core guarantees as a much larger enterprise data platform: a clean separation between raw and modeled data, automated data-quality enforcement, daily unattended orchestration, multi-level documentation, and a governed, trustworthy path all the way through to a business-facing dashboard.

The project's real strength is not any single component — it is that every layer reinforces the others. The medallion architecture (Section 1) makes the testing strategy (Section 2) possible, because tests can target a specific, well-defined layer instead of an undifferentiated blob of logic. The documentation (Section 3) makes the whole system legible to anyone who did not build it. And the dashboard (Section 4) is only trustworthy because of everything that happened upstream of it.

5.2 Where InsightStream Goes Next

- Incremental loading: move from full daily reloads to dbt incremental models and Snowflake Streams/Tasks, reducing both run time and warehouse compute cost as data volume grows.
- CI/CD for the pipeline: wire the dbt project into GitHub Actions so every pull request automatically runs dbt build and dbt test against a staging schema before merging.
- Alerting: connect Airflow task failures to Slack or email notifications, closing the loop between “a test failed” and “someone actually knows about it” in real time.
- Deeper BI: introduce DAX-based calculated measures (e.g., month-over-month growth, customer lifetime value) and row-level security so different stakeholders see only the data relevant to them.
- Forecasting: extend the seasonality pattern observed in Section 4 into an actual demand-forecasting model, feeding inventory and staffing decisions automatically rather than relying on a human reading a chart.

Closing thought

The most common failure mode for analytics projects is not a lack of ambition — it is skipping the unglamorous foundation in a rush to reach the dashboard. InsightStream took the opposite approach: architecture, testing, and documentation came first, and the dashboard in Section 4 is simply the visible tip of everything described in the sections before it.

That ordering is the real takeaway of this report, and the principle worth carrying into whatever this pipeline grows into next: build the foundation so solidly that the dashboard becomes the easy part.